



Informed search algorithms

Lecturer: Nguyen Thanh Binh





Outline

- *Relaxed problems*
 - *Local search algorithms*
- 



Relaxed problems

- *A problem with fewer restrictions on the actions is called a relaxed problem*
- *The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem.*
- *If the rules of the 8-puzzle are relaxed so that a tile can move anywhere, the heuristic function $h_1(n)$ gives the shortest solution*



Relaxed problems

- *If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution.*
- *The original problem can be decomposed into many independent subproblems by the relaxed rules.*



Learning heuristics

- *A heuristic function $h(n)$ is supposed to estimate the cost of a solution beginning from state at node n .*
 - *An agent can construct such a function by devising relaxed problems for which an optimal solution can be found easily.*
 - *Another option is to learn from experience.*
- 



Learning heuristics

- *For example, we solve a lot of 8-puzzles and get an optimal solution for each problem.*
- *Each such solution can provide examples for which one can learn an heuristic function $h(n)$.*
- *From these examples, one can construct a function $h(n)$ by an inductive learning that can predict solution costs from other states that arise during search (using neural networks, decision trees,...)*





Learning heuristics

- *Inductive learning methods work best when supplied with features of state that are relevant to its evaluation.*
- *For example: the feature “numbers of misplaced tiles” $\rightarrow x_1(n)$. Take 100 randomly generated 8-puzzle configurations and gather statistics on their actual solution costs.*





Learning heuristics

- *One may find that when $x_1(n)$ is 5, the average solution is around 14,...*
- *A second feature $x_2(n)$: “the number of pairs of adjacent tiles that are adjacent in the goal state as well”.*
- *Combine the statistical results from $x_1(n)$ and $x_2(n)$ to predict $h(n)$: $h(n) = c_1 \cdot x_1(n) + c_2 \cdot x_2(n)$*





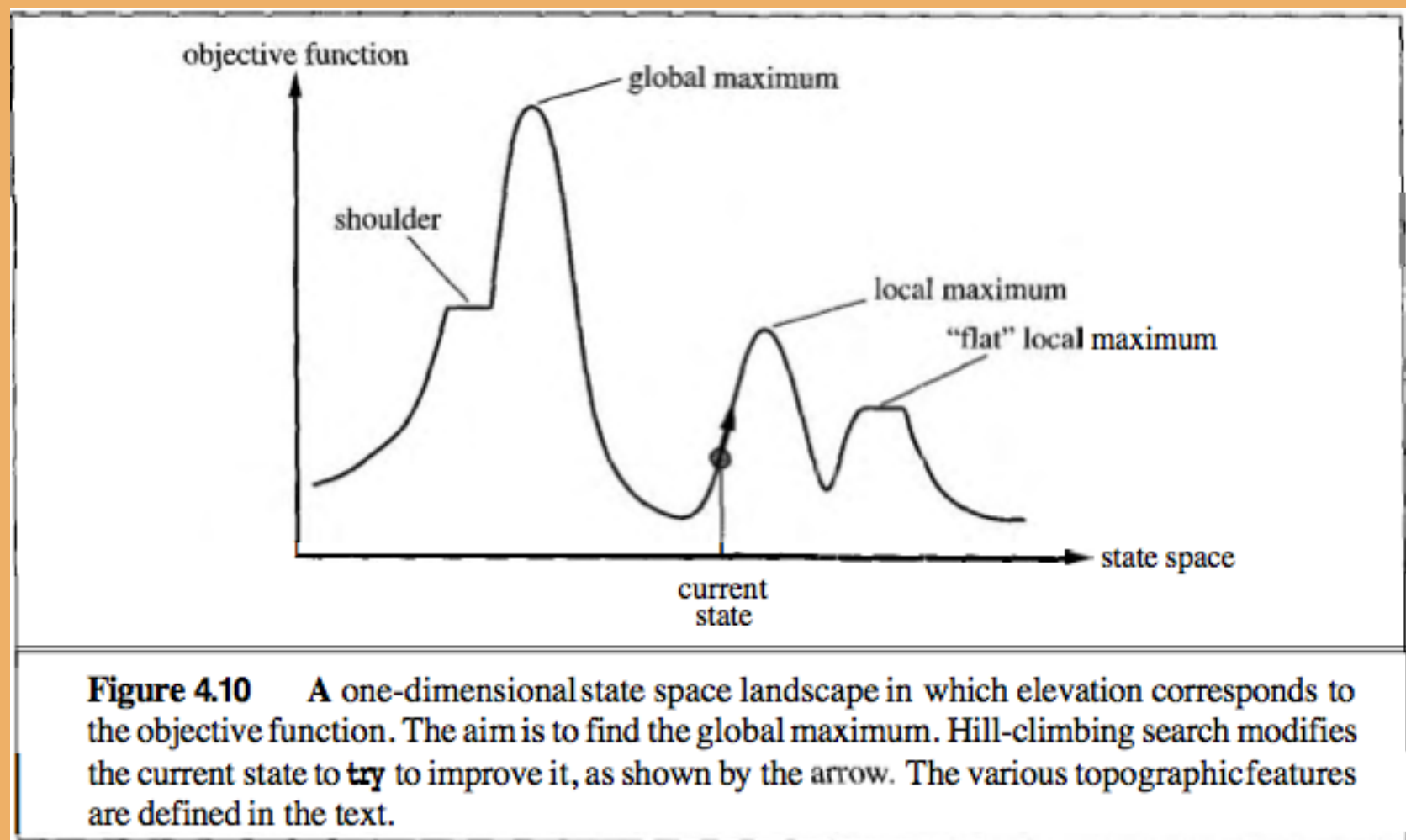
Local search algorithms

- *Use a single current state and generally move only to neighbors of that state.*
- *Not systematic.*
- *Two key advantages:*
 - *use very little memory- usually a constant amount.*
 - *often find reasonable solutions in large or infinite state spaces for which systematic algorithms are unsuitable.*



Local search algorithms

- Use to solve pure optimization problems.



Hill-climbing algorithm

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                     neighbor, a node

  current  $\leftarrow$  MAKE-NODE(INITIAL-STATE[ problem ])
  loop do
    neighbor  $\leftarrow$  a highest-valued successor of current
    if VALUE[neighbor]  $\leq$  VALUE[current] then return STATE[current]
    current  $\leftarrow$  neighbor
```

Figure 4.11 The hill-climbing search algorithm (**steepest ascent** version), which is the most basic local search technique. At each step the current node is replaced by the best neighbor; in this version, that means the neighbor with the highest VALUE, but if a heuristic cost estimate h is used, we would find the neighbor with the lowest h .

Simulated annealing search

- *A hill-climbing algorithm can get stuck on a local maximum. Incomplete and fast.*
- *A purely random walk (moving to a successor chosen uniformly at random from the set of successors. Complete but extremely inefficient.*
- *Combine hill-climbing algorithm with a random walk in some ways that make efficiency and completeness.
-> simulated annealing.*

Simulated annealing search

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                    next, a node
                    T, a "temperature" controlling the probability of downward steps

  current  $\leftarrow$  MAKE-NODE(INITIAL-STATE[problem])
  for t  $\leftarrow$  1 to  $\infty$  do
    T  $\leftarrow$  schedule[t]
    if T = 0 then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow$  VALUE[next] - VALUE[current]
    if  $\Delta E > 0$  then current  $\leftarrow$  next
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$ 
```

Figure 4.14 The simulated annealing search algorithm, a version of stochastic hill climbing where some downhill moves are allowed. Downhill moves are accepted readily early in the annealing schedule and then less often as time goes on. The *schedule* input determines the value of *T* as a function of time.